

Data Connectivity & Architecture Review

Oxford Guardians Platform — mapping what should be joined up, and isn't

Prepared 25 June 2026

Scope: full codebase (~170 pages)

Status: prototype

This review traces how data actually flows between the platform's pages, identifies sections that were built at different stages and never wired together, recommends both obvious and non-obvious links, and assesses whether joining them up would make pages too heavy — with a concrete strategy for keeping things stable.

CONTENTS

1. **How the platform is built today** — the one-database, many-views model
2. **Broken links: things that *should* connect but don't** — staged work left unjoined
3. **Non-obvious connections worth making** — higher-effort, high-value joins
4. **Weight & stability: will connecting things break pages?** — and how to split them
5. **Appendix** — data-store inventory, orphan pages, quick-win checklist

Executive summary

The platform is in good shape architecturally: almost all data goes through a **single shared data layer** (`old_version/assets/db.js` , ~1,900 lines) that reads and writes browser `localStorage` , and each HTML page is a thin "view" on top of it. That is the right pattern and it means most fixes are small and central.

The disconnects you're feeling are real, and they fall into three buckets:

- **The document library has blind spots.** The aggregator that powers the Documents pages only looks at *some* of the places documents are uploaded. Staff documents, parent documents, private-fostering documents, and the entire *compliance* document set (DBS, ID, references) are invisible to it.
- **Two parallel "host / homestay" systems exist.** A rich, wired-up *Hosts* system and a thinner, partly-static *Homestays* system were built at different times and overlap heavily.
- **Many records are joined by name, not by ID.** Concerns, finances, excats, schools and parents reference people by typed-in name strings instead of stable IDs — so the system can't reliably show "everything about this one person" in one place.

None of these require a rewrite. The biggest single win is making the **document aggregator** the one true source for every uploaded file, and adding **ID links** to the four name-based record types. On weight: the heaviest pages today (`finances-tracker.html` ~1,350 lines, `host-profile.html` ~1,315 lines) are already near the limit of what's comfortable in a single inline-script page, so Section 4 recommends a tab/lazy-render approach rather than piling more onto them.

1 · How the platform is built today

Two things are worth knowing before the findings make sense.

1.1 One database, many views

Everything runs client-side. `db.js` defines ~30 `localStorage` “tables” (students, hosts, drivers, staff, parents, concerns, incidents, exeats, placements, welfare logs, mental-health cases, finance events, and more), seeds them with demo data on first load, and exposes one global API object, `window.OG_DB`, with ~80 functions. Every page includes `db.js` and calls into `OG_DB` to read/write. **This is the single source of truth and the right place to make most of the fixes below.**

1.2 Records link to each other two different ways

This split is the root cause of most of what feels “disconnected”:

✓ Linked by stable ID (good)

These can reliably find each other:

```
driver.linkedStudents[]
host.linkedStudents[]
parent.linkedStudentIds[]
welfareLog.studentId  mhCase[studentId]
visitReport.hostId    hsChecklist.hostId
trainingLog.hostId    todo.placementId
alert.personId
```

✗ Linked by typed-in name (fragile)

These break if a name is spelt differently or changes:

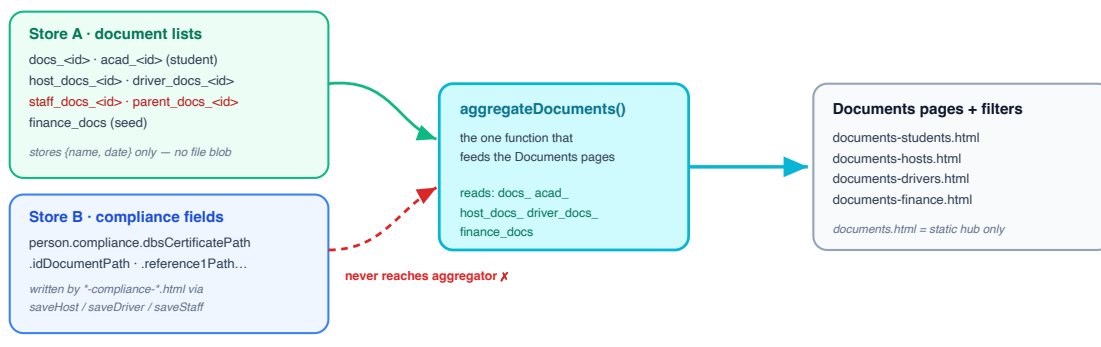
```
concern.subjectName  financeEvent.student
financeDoc.person     exeat.studentName
student.school        student.parent1Name (inline)
```

Example already in the seed data: `finance-data.js` bills students named “Lucia Herrera” and “Aarav Mehta”, who don’t exist in the student database at all — a name-based link with nothing on the other end.

1.3 The document model — three stores that don’t talk

The same person’s documents can live in up to three unconnected places:

WHERE FILES ARE UPLOADED



The aggregator only sees part of Store A and never sees Store B. Anything in red is invisible to the Documents filters.

2 · Broken links: things that should connect but don't

These are gaps where two pieces clearly belong together — they were just built at different stages and never joined. Ordered by impact.

#	DISCONNECT	WHERE	SEVERITY
D1	Document aggregator ignores staff, parent & private-fostering uploads	db.js · documents-*.html	High
D2	Compliance documents (DBS/ID/references) never appear in the document library	*-compliance-*.html	High
D3	Two parallel systems: <i>Homestays</i> vs <i>Hosts</i>	homestay-*.html vs host-*.html	High
D4	Concerns aren't attached to the student/host they're about	concerns-*.html · student-profile	Med
D5	Compliance alerts can't be cleared from the document library (separate store)	db.js generateComplianceAlerts	Med
D6	<code>documents.html</code> hub is static — disconnected from the live aggregator	documents.html	Low
D7	Root <code>index.html</code> redirects to a hard-coded local file path	index.html:602	Med

D1 — The Documents pages have blind spots

`aggregateDocuments()` in `db.js` pulls from `docs_` `acad_` `host_docs_` `driver_docs_` `finance_docs`, but uploads are *also* written to:

`staff_docs_<id>` (staff-profile.html) `parent_docs_<id>` (parent-profile.html) `og_pf_documents` (private-fostering-documents.html)

Effect: exactly the symptom you described — a file uploaded on a newer section never shows up in the Documents pages or their filters. **Fix:** add three short blocks to `aggregateDocuments()` (one per source). ~20 lines, fully central. This is the single highest-value fix in the report.

D2 — Compliance documents live in a different world

The compliance sub-pages (`driver-compliance-dbs.html`, `host-compliance-identity.html`, etc.) save uploads as *fields on the person record* — `compliance.dbsCertificatePath`, `idDocumentPath`, `reference1Path`, `safeguardingCertificatePath` — via `saveHost/saveDriver/saveStaff`. The document library (Store A) never sees them.

Effect: the most safeguarding-critical files (DBS, right-to-work, references) are *invisible* in the Documents browser. They only surface in the SCR pages. **Fix:** teach `aggregateDocuments()` to also walk each person's `compliance.*Path` fields and emit a row per non-empty path (tagged sub-type "Compliance"). Then the library becomes complete and D5 partly resolves too.

D3 — “Homestays” and “Hosts” are two systems for one concept

There is only one data table (`og_hosts`), but two UIs sit on top of it:

HOSTS CLUSTER (WIRED, RICH)	HOMESTAYS CLUSTER (MIXED / PARTLY STATIC)
<code>hosts.html</code> → <code>host-profile.html</code> (1,315 lines, full records, visit reports, compliance)	<code>homestays.html</code> → <code>homestay-profile.html</code> (83-line static stub)
<code>host-create.html</code> writes via <code>saveHost</code>	<code>homestay-create.html</code> reads hosts but several siblings are static

Within the Homestays cluster, some pages *do* write to the shared DB (`homestay-visit-report.html` → `saveVisitReport` , `homestay-hs-checklist.html` → `saveHsChecklist` , `homestay-incident-create.html` → `saveIncident`) while others are dead stubs (`homestay-visit-create.html` , `homestay-assessment-create.html` , `homestay-accident-create.html` , `homestay-approve.html` , `homestay-profile.html`).

The trap: the dashboard's *Homestays* tab leads users into the thin cluster and its static profile stub, while the real, data-rich host record lives one click away in `host-profile.html` . **Fix:** pick *one* as canonical (recommend the Hosts cluster — it's where the data is), point the dashboard tab and `homestay-profile.html` at `host-profile.html?id=...` , and fold the genuinely useful homestay actions (visit/checklist/incident) in as tabs on the host profile.

D4 — Concerns float free of the people they concern

Concern records carry `subjectName` + `subjectRole` but no `subjectId` . So a student/host profile can't list “open concerns about this person,” and a concern can't deep-link back to a profile. **Fix:** add `subjectId` to the concern schema (and a picker on the create form), then show a concern strip on the relevant profile. Foundation for the unified timeline in §3.

D5 — Uploading a document doesn't clear the matching alert

`generateComplianceAlerts()` checks `compliance.*Path` fields. Because the document library writes to a different store (D2), uploading “DBS certificate” there leaves the “No DBS on file” alert standing. Unifying the stores (D2) makes alerts respond to either path.

D6 / D7 — Smaller but worth noting

D6: `documents.html` is a static navigation hub with no data calls; the real work is in the four `documents-*.html` pages. Either retire it or have it render live counts from the aggregator so it stops looking like “the old documents section.”

D7: the root `index.html` login redirects to `location.href = '/Users/tamid/Documents/GitHub/.../admin-dashboard.html'` — an absolute path to your own machine. It will 404 on any other computer or host. Change to a relative path (`old_version/admin-dashboard.html`).

3 · Non-obvious connections worth making

These aren't bugs and nobody would flag them as "broken" — but wiring them up is what turns a set of separate screens into a coherent platform. Each one is enabled by giving records stable IDs instead of names (§1.2).

C1 — A single safeguarding/welfare timeline on the student profile

Today a student's story is scattered across welfare logs, mental-health case + comms, concerns, incidents, and exeats — each a separate screen, some linked by ID and some by name. Merging them into one date-sorted timeline on `student-profile.html` would let a coordinator see everything that happened to a child in one view. Needs: `concern` `subjectId` (D4), `incident` `studentId`, and the §2 fixes.

C2 — Finance ↔ the people it's about

`finance-data.js` events reference `student` as a name, and `finance_docs` reference `person` as "Tanaka Family." Key them to `STU.../DRV.../HST...` IDs instead and you unlock: a per-student cost panel on the profile, driver billing statements that hang off the driver record, and finance docs that finally flow into the document library (D1).

C3 — Schools ↔ students, logs & services

Students store `school` as a name that happens to match `school.name`; there's already a schools table with a per-school services matrix and school logs. Add a `schoolId` to students and `school-profile.html` can show enrolled students, recent school logs, and which services apply — instead of being a near-empty page. Also resolves the redundancy between `schools.html`, `school-directory.html`, `school-liaison.html` and `school-profile.html`, which currently overlap.

C4 — Parents: one source of truth

Parent contact details live *twice*: inline on the student (`parent1Name/Email/Phone`) and as full records in `og_parents` (with `linkedStudentIds`). These can silently drift apart. Make the student's parent fields reference parent records (or auto-create a parent record from them) so the parent portal, parent liaison logs, and the student profile all agree.

C5 — Logistics: exeats ↔ transports ↔ placements

There are three overlapping logistics stores — `og_exeats`, `og_transports` (+ travel todos) and `og_placements` (+ todos) — each generating its own checklist. They describe the same student journeys. Cross-reference them by `studentId` and an exeat could auto-spawn its transport leg and homestay placement instead of re-entering the same trip three times. The *Exeat Calendar.xlsx* / *exeat workbook.xlsx* in the repo suggest an import path that was never wired in.

C6 — Host monitoring feeds compliance status automatically

Visit reports already nudge `host.annualVisitDue` when an annual review is saved — extend the same idea so H&S checklists and training logs feed the compliance alert generator and SCR. Then "host monitoring up to date" becomes a computed fact, not a manually-typed status field.

4 · Will connecting things make pages too heavy — and how to split them

Short answer: **connecting the data won't bloat anything** — it all flows through `db.js` and stays out of the page. The risk is on a few *view* pages where we'd be tempted to render everything at once. Two pages are already near the comfortable limit.

4.1 Where the weight actually is

PAGE	LINES	RISK IF WE ADD MORE
<code>finances-tracker.html</code>	1,347	Watch already a mega-page
<code>host-profile.html</code>	1,315	Watch target for homestay merge (D3)
<code>revenue.html</code> / <code>archived-revenue.html</code>	~950 / ~870	OK but duplicated logic
<code>reports.html</code>	877	OK
<code>student-profile.html</code>	~820	Watch target for timeline (C1)

The real fragility isn't line count — it's that **each page re-implements its own rendering in one big inline `<script>`**. Adding a timeline or a documents panel to a 1,300-line page means more inline code to keep in sync, and more that renders on every load.

4.2 The actual breakage risk: `localStorage` limits

Browsers cap `localStorage` at roughly **5–10 MB per origin**. **Good news:** the upload widgets today store only `{name, date}` metadata — *not* the file bytes — so you're nowhere near the limit yet. But the moment a "real" build stores actual files (base64 data URLs) in `localStorage`, a handful of PDFs will blow the quota and writes will start throwing `QuotaExceededError` — which the current `try/catch` blocks swallow silently, so data would vanish with no error shown. **This is the most likely way the platform "breaks" as it grows.** Keep file *bytes* out of `localStorage`; store metadata + a reference, and move to IndexedDB or a real backend before storing binaries.

4.3 Recommended structure to keep things light & stable

1. **Keep `db.js` as the only data layer.** All the §2/§3 joins go here as functions (e.g. `getTimelineForStudent(id)`, `getDocumentsForPerson(id)`). Pages stay thin. This is already your pattern — lean into it.
2. **Render heavy profiles in lazy tabs.** On host/student profiles, don't build every section on load. Render the active tab only and build the others on first click. This is what lets you add the timeline (C1), documents panel (D1/D2) and finance panel (C2) to a profile *without* the page getting slower.
3. **Keep the Documents library split by owner (it already is)** — students / hosts / drivers / finance — but feed all four from *one* complete `aggregateDocuments()` (after D1+D2). Splitting the *view* is good; splitting the *source* is what caused the blind spots.
4. **Extract shared rendering into a `render-helpers.js`.** Table-builders, the upload widget, date formatting and the doc-list renderer are copy-pasted across many pages. One shared module shrinks every page and removes the "fix it in 12 places" problem (e.g. revenue / archived-revenue duplication).
5. **De-duplicate the parallel clusters** (Homestays→Hosts D3; school pages C3; revenue/archived-revenue) rather than maintaining two of each.

4.4 What to split out into its own section

When a profile tab grows past a screenful of distinct workflow, give it its own page reached by `?id=` (the platform already does this well — `student-risk.html?id=`, `student-health.html?id=`). Good candidates to *keep separate* rather than

inline:

- Finance per student → a dedicated `student-finance.html?id=` tab, not inline on the profile (finance logic is heavy).
- The full document set per person → reuse the documents pages filtered by `personId`, linked from the profile, rather than re-rendering the list inline.
- The safeguarding timeline (C1) → render inline but */lazy*, and cap it to the most recent N with a "view all" link to a full-history page.

5 · Appendix

5.1 Quick-win checklist (ordered by value ÷ effort)

#	ACTION	EFFORT	UNLOCKS
1	Add staff / parent / private-fostering sources to <code>aggregateDocuments()</code>	~20 lines, central	Fixes the main symptom (D1)
2	Add <code>compliance.*Path</code> walk to the aggregator	~25 lines, central	D2 + D5
3	Fix root <code>index.html</code> absolute redirect → relative	1 line	D7 (portability)
4	Point Homestays tab + <code>homestay-profile</code> at the Hosts system	Small	D3
5	Add <code>subjectId</code> to concerns + picker on create form	Medium	D4 → C1
6	Add <code>schoolId</code> / <code>studentId</code> keys to schools, finance, exeats	Medium	C2 C3 C5
7	Extract <code>render-helpers.js</code> ; de-dup revenue/school clusters	Medium	§4 stability

5.2 Document storage map (the heart of the problem)

STORAGE KEY	WRITTEN BY	IN AGGREGATOR?
<code>docs_<id></code> (student general)	student-profile.html	Yes
<code>acad_<id></code> (student academic)	student-profile · student_academics	Yes
<code>host_docs_<id></code>	host-profile.html	Yes
<code>driver_docs_<id></code>	driver-profile · drivers.html	Yes
<code>finance_docs</code>	db.js seed	Yes
<code>staff_docs_<id></code>	staff-profile.html	No
<code>parent_docs_<id></code>	parent-profile.html	No
<code>og_pf_documents</code>	private-fostering-documents/-notification	No
<code>compliance.*Path</code> (DBS, ID, refs...)	all *-compliance-*.html	No

5.3 Notable orphan / stub pages

Reached only via JS navigation or not at all. Several compliance sub-pages are reached dynamically (expected); the static stubs below are the ones tied to the parallel-systems issue (D3).

<code>homestay-profile.html</code> (stub)	<code>homestay-approve.html</code> (static)	<code>homestay-visit-create.html</code> (static)
<code>homestay-assessment-create.html</code> (static)	<code>homestay-accident-create.html</code> (static)	
<code>student_academics.html</code> (underscore naming)	<code>concern-escalation.html</code>	<code>low-level-concerns.html</code>
<code>mental-health-comms.html</code>	<code>reference.html</code>	